

William Blum
Principal Development
Lead
Microsoft

Lambda Days 2017,
Kraków

Building a cloud service with F#

Structure of the talk

- My journey to FP
- Project Springfield
- Why and how we built it in F#

My FP journey

Caml Light


$$e := x \mid \lambda x. e \mid e e$$


Executive Bloggers	Visual Studio	Application Lifecycle Management	Languages	.NET	Platform Development
--------------------	---------------	----------------------------------	-----------	------	----------------------

.NET Blog

A first-hand look from the .NET engineering teams

Project Springfield: a Cloud Service Built Entirely in F#

December 13, 2016 by Visual FSharp Team [MSFT] // 17 Comments



This post was written by **William Blum**, a Principal Software Engineering Manager on the Springfield team at Microsoft Research.

Earlier this year, Microsoft announced a preview of Project Springfield, one of the most sophisticated tools Microsoft has for rooting out potential security vulnerabilities in software. Project Springfield is a fuzz testing service which finds security-critical bugs in your code.

One of the amazing things about Springfield is that it's built on top of Microsoft's development platforms – F#, .NET, and Azure! This post will go over some of the what, why, and how of Project Springfield, F#, .NET, and Azure.

What is Project Springfield?

Project Springfield is Microsoft's unique fuzz testing service for finding security critical bugs in software. It helps you quickly adopt practices and technology from Microsoft. The service leverages the power of the Azure cloud to scale security testing using a suite of security testing tools from Microsoft. It's currently in preview, and you can sign up at [Project Springfield](#) if you want to give it a try!



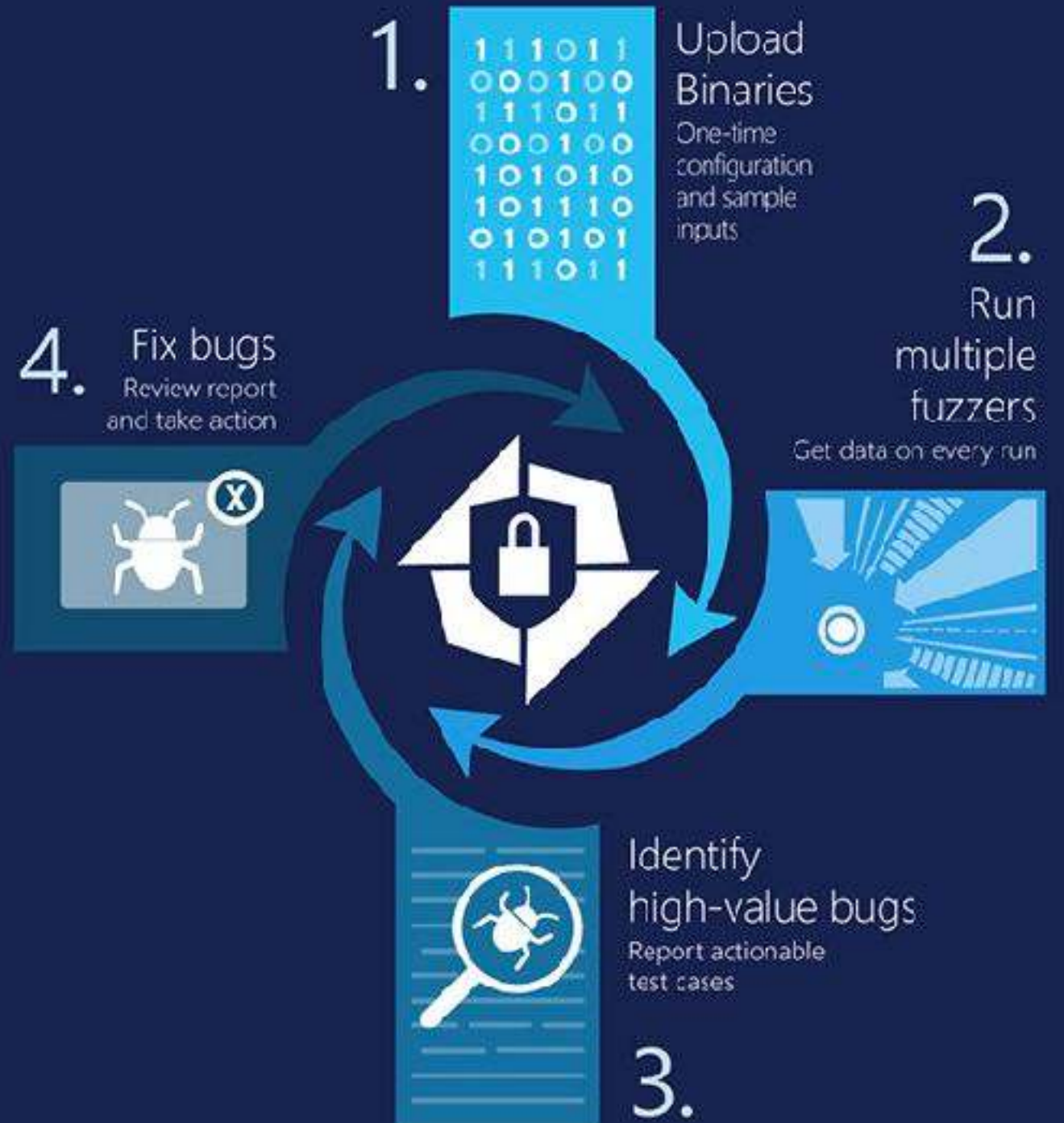


Microsoft Research NExT

- Natick Undersea data center
- Skype translator
- Cognitive services API for NLP and Image recognition
- DNA storage “storing petabytes of data in a shoe box”
- Holoportation teleportation via HoloLens
- Springfield

Project Springfield

Security testing in the cloud running Microsoft fuzzing technologies



Springfield

Springfield

DocumentationSettings1@outlook.com

Job Results

Start Date

End Date

Download Selected Items

☒	Created ▾	Severity ▾	Type ▾	Hash	CallStack ▾	InputCount ▾	Action
☐	2015/11/04	Crashes	Other Exception	1330571949	ntdll!RtlUnhandledExceptionFilter+0x29F	10	Download Details
☐	2015/11/04	Crashes	Read Access Violation	169607970	ntdll!RtlAllocateHeap+0x9F4	10	Download Details
☐	2015/11/04	FirstChanceExceptions	Breakpoint	1327762509	ntdll!RtlUnhandledExceptionFilter+0x29F	10	Download Details
☐	2015/11/04	Crashes	Write Access Violation	520963147	DemoFuzz/main+0xDF	10	Download Details
☐	2015/11/04	Crashes	Null	169345826	ntdll!RtlAllocateHeap+0x9F0	10	

Contact us

Privacy and Cookies

Terms of use

Code of Conduct

Trademarks

Third Party Notices

© Microsoft Corporation.

Microsoft

Springfield Dev Team

Started 2 years ago

Initially 3 devs - Now 6 F# devs

Team background:

- C/C++, C#, Java
- Some previous F# experience
- All interested in learning more about FP!



Springfield Tech Stack

F#

- Frontend, Backend, Client, Operation, Core API

Azure

- Compute, Network, Storage, App Services
- Service Fabric

TypeScript

- Web client

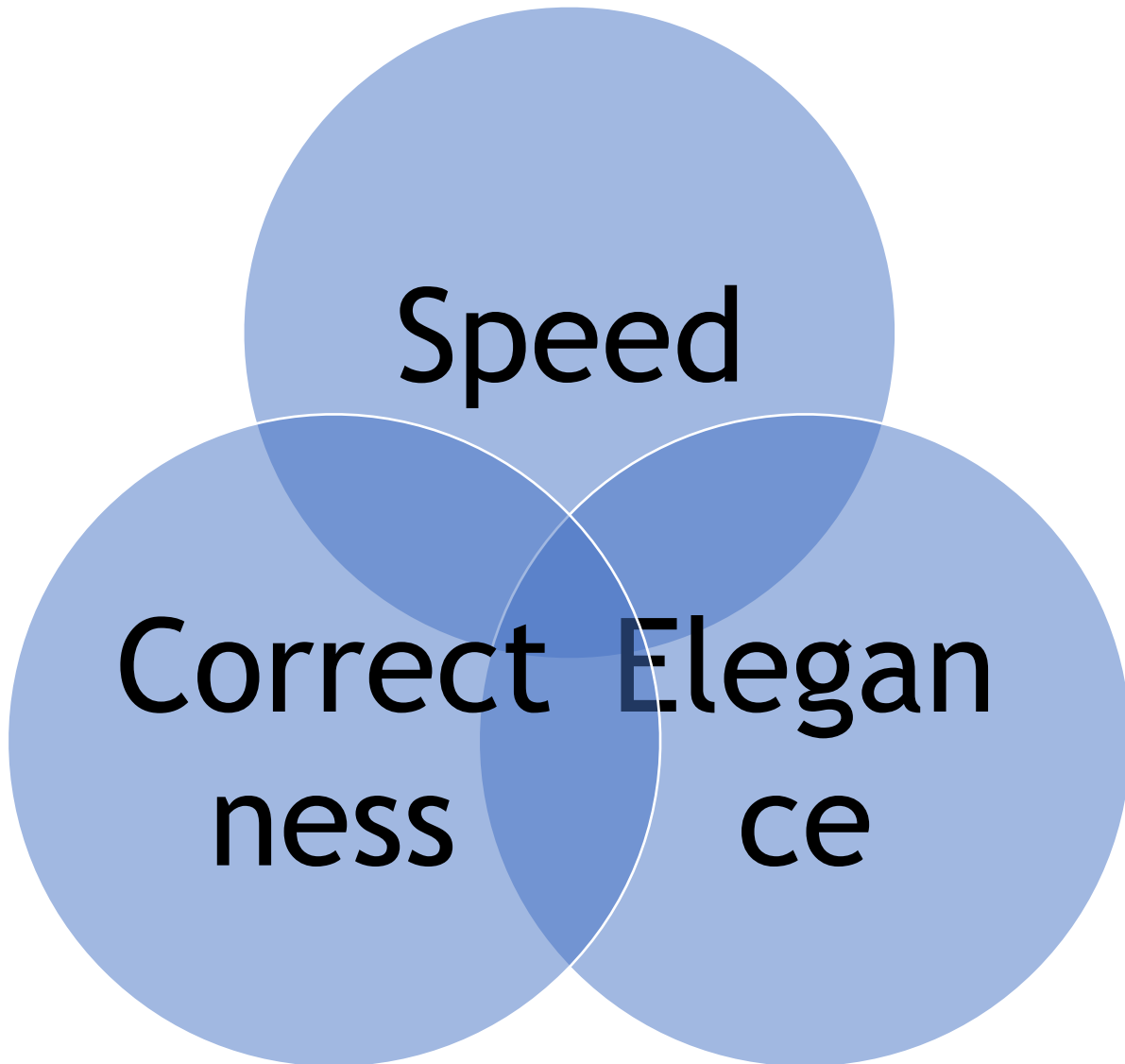
PowerShell

- Deployment

Fuzzing tools

- Tools from Office Team and MSR
(written in C# and C++)

Why F#?



Fast prototyping
Fast onboarding
=> MVP in 3 months



Lean, focused team, no
dedicated test resource.



Fun programming!

Functional/F# benefits

Speed

- Concise
- Intentional
- Interactive REPL
- Tooling/IDE support
- C#/.NET Interop
- .NET libraries
- Community support

Correctness

- Strong type checking
- Immutability
- option type
- printf type checking
- Theoretical foundation
- Unit of measure

Elegance

- Lambda abstractions
- Type inference
- Let-polymorphism
- Recursion
- Pattern matching
- Algebraic types
- Type providers
- Computational expressions
- Async workflows
- Active patterns

“Why bother with F# now that C# has lambdas?”

Feature	F#	C#
Type inference	1.0 (built in)	3.0 (2007)
Lambda	1.0 (built in)	3.0 (2007)
REPL	1.0 (built in)	6.0
Asynchronous programming	2007	5.0 (2012)
Immutability by default	1.0 (built in)	Not planned
Safe printf	1.0 (built in)	Not planned
Algebraic types, option type	1.0 (built in)	Not planned
null safety	1.0 (built in)	Non-nullable? 7.0
Type providers	3.0, 2012	Not planned
Computational Expression	2.0	Not planned
Local functions, Tuples, records, pattern matching	1.0 (Built in)	7.0?

Correctness: option type

“NULL pointers are the biggest problem in C/C++, at least in our code. The dual use of a single value as both a flag and an address causes an incredible number of fatal issues. C++ references should be favored over pointers (...) Perform NULL checks when pointers are turned into references, then you can ignore the issue thereafter. There are a lot of deeply ingrained game programming patterns that are just dangerous, but I'm not sure how to gently migrate away from all the NULL checking.”

John Carmack

http://www.gamasutra.com/view/news/128836/InDepth_Static_Code_Analysis.php

```
type 'a Option =  
| None  
| Some of 'a
```

```
match o with  
| None ->  
| Some v ->
```

Correctness: typed-checked printf

“Printf format errors were the second biggest issue in our codebase, (...) passing an idStr almost always results in a crash, but annotating all our variadic functions with kills this problem dead. There were dozens of these hiding in informative warning messages that would turn into crashes (...), which is also a comment about how the code coverage of our general testing was lacking.”

John Carmack

http://www.gamasutra.com/view/news/128836/InDepth_Static_Code_Analysis.php

```
open Microsoft.FSharpLu.TraceLogging
Trace.info "Created %d machines for %s at %0"
          machines name System.DateTime.Now
```


Recruiting: FP attracts talents!

Growing adoption

- Scala used at Twitter, Amazon
- Elixir growing popularity in the Bay Area
- Elm and PureScript are growing for web front-end
- Haskell/Ocaml at Facebook, Jane Street Capital
- Apple Swift taking over Objective C
- F# at Jet.com

Hired 4 programmers because of F#:

- 3 C#/Java dev who wanted to do F# full time
- 1 C#, C/C++ who show interest in our team because of F# blog post

Domain types expressed with Records + ADT + Discriminated Unions

```
type VmParameters =  
    {  
        customerId : CustomerId  
        jobId : JobId  
        runId : JobRunId  
        submitDate : System.DateTime  
        appInsightsKey: string option  
        vmPurposeParameters : VmPurposeParameters  
        version : Version  
    }
```

```
type VmPurposeParameters =  
    | CustomerVM of CustomerVmParameters  
    | FuzzingVM of FuzzingVmParameters
```

```
type VmSize =  
    | ExtraSmall  
    | Small  
    | Medium  
    | Large  
    | ExtraLarge
```

Reliability via agent-based programming

```
type Transition<'s, 't> =  
| RetryIn of System.TimeSpan  
| Goto of 's  
| Return of 't
```

```
type Channel<'m> =  
{  
    send : 'm -> Async<unit>  
    sendIn : 'm -> System.TimeSpan -> Async<unit>  
}
```

```
type Agent<'s, 't, 'm> =  
{  
    transition : Channel<'m> -> 's  
                -> Async<Transition<'s, 't>>  
    serializeState : 's -> 'm  
    persist : 'm -> System.TimeSpan -> unit  
    channel : Channel<'m>  
}
```

```
'a -> ('b -> Async<'c>)  
let createJob (channel:Agent.Channel<_>) = function  
    .... | JobCreationRequest ->  
    ..... async [  
    .....     let! name = createAzureVM ()  
    .....     return Goto <| WaitForVM(name)  
    ..... ]  
    .... | WaitForVM name ->  
    ..... async {  
    .....     let! ready = isVMDeployed name  
    .....     if ready then  
    .....         return Goto <| JobCreated(name)  
    .....     else  
    .....         return RetryIn 5<s>  
    ..... }  
    .... | JobCreated name ->  
    ..... async {  
    .....     do! NotifyUserVMIReady()  
    .....     return Return name  
    ..... }
```

Resumable monad

```
let myServiceApi =  
  resumable {  
    let! machineName = resumable { return Environment.getMachineName () }  
    let! requestId = resumable { return Environment.provisionVM machineName }  
  
    let! vmready = resumable {  
      while not <| Environment.vmRequestSucceeded requestId do  
        printfn "Waiting for cloud request to complete..."  
        System.Threading.Thread.Sleep(1000)  
      return true  
    }  
  
    printfn "Request completed for machine %s!" machineName  
    return machineName, requestId, vmready  
  }  
}
```

<http://blumu.github.io/ResumableMonad/>

JSON serialization

```
type 'a Tree =  
  Leaf of 'a | Node of 'a Tree * 'a Tree  
  
let x = Node (Node((Leaf 1), (Leaf 4)), Leaf  
6)
```

Default.serialize x =

```
"{ \"Case\": \"Node\", \"Fields\":  
[ { \"Case\": \"Node\", \"Fields\":  
[ { \"Case\": \"Leaf\", \"Fields\": [ 1 ] },  
{ \"Case\": \"Leaf\", \"Fields\":  
[ 4 ] } ] }, { \"Case\": \"Leaf\",  
\"Fields\": [ 6 ] } ] }"
```

Compact.serialize x =

```
"{ \"Node\": [ {  
  \"Node\": [ { \"Leaf\": 1 },  
    { \"Leaf\": 4 } ] },  
  { \"Leaf\": 6 } ] }"
```

<https://github.com/Microsoft/fsharpplu/wiki/fsharpplu.json>

F# Type Providers

```
open System.Linq
open Microsoft.FSharp.Linq
open Microsoft.FSharp.Data.TypeProviders

type NorthwndDb =
    SqlConnection<ConnectionString = @"AttachDBFileName = 'C:\project

let db = NorthwndDb.GetDataContext()

let customerNames =
    query { for c in db. do
        where (c.Ci
        select c.Con
        AlphabeticalListOfProducts
        Categories
        CategorySalesFor1997s
        property
        NorthwndDb.ServiceTypes.Simple
        AlphabeticalListOfProducts:
        System.Data.Linq.Table<Northwnd
```

F# type providers in action: <https://blogs.msdn.microsoft.com/dsyme/2013/01/30/twelve-f-type-providers-in-action/>

Statically checking data with Type Providers

```
Provisioning.fs x
508
509 let g (x:System.Guid) =
510     .... CustomerAccountTemplateParameters.AccountId(x)
511 let s (x:string) =
512     .... CustomerAccountTemplateParameters.Location(x)
513
514 let armTemplateParameters =
515     .... CustomerAccountTemplateParameters.Parameters(
516         .... accountId = s customerId,
517         .... location = s location,
518         .... adUserObjectId = s adUserObjectId
519     ....)
520
521 let tags = [ "displayName", sprintf "Customer resources
522             .... "customerId", customerId.ToString()
523             .... "purpose", "Customer"
524             .... ] @ (commonTags provision)
525
526 let! r = Azure.ResourceGroup.
```

```
CustomerAccount.param.dev.json x
3     .... "contentVersion": "1.0.0.0",
4     .... "parameters": {
5         .... "accountId": {
6             .... "value": "2C91C585-A41E-46E6-B9C1-96
7         .... },
8         .... "location": {
9             .... "value": "West US"
10        .... },
11        .... "tenantId": {
12            .... "value": "tenant id"
13        .... },
14        .... "adUserObjectId": {
15            .... "value": "ad user id"
16        .... },
17        .... "keyVault": {
18            .... "value": "sfkvwiblumservice"
19        .... }
```

```
1 open FSharp.Data
2 type CustomerAccountTemplateParameters =
3     JsonProvider<"CustomerAccount.param.dev.json", InferTypesFromValues=true>
```

Azure and Type Provider



SwaggerProvider by Sergey Tihon. **3.14K** downloads
F# Type Provider for Swagger

```
open SwaggerProvider
Type Azure =
    <SwaggerProvider>
    <URL> "https://raw.githubusercontent.com/Azure/azure-rest-api-specs/master/arm-compute/2016-03-30/swagger/compute.json">
    <...>
let a = new Azure(headers: authHeader)
a.Vi
```

VirtualMachineExtensionImagesGet
VirtualMachineExtensionImagesListTypes
VirtualMachineExtensionImagesListVersions

Azure.VirtualMachineExtensionImagesGet(location: string, publisherName: string, type: string, version: string, subscriptionId: string) : Azure.VirtualMachineExtensionImage

```
a.VirtualMachinesListAll(sdkVersion, subscriptionId).Value  
|> Seq.countBy (fun vm -> vm.Properties.HardwareProfile.VmSize)
```


Asynchronous programming with F#

- Springfield code entirely async
- C# pitfall avoided
- Implemented as a computational expression, not a compiler trick

```
'a -> 'b -> Async<unit>
let deleteVhd (storage:BlobStorageContext) path =
    .... async {
        ..... let container = storage.client.GetContainerReference("container")
        ..... let blob = container.GetBlobReference(path)
        ..... let! r = blob.DeleteIfExistsAsync().AsAsync
        ..... if not r then
        .....     invalidOp "VHD blob does not exist"
    .... }
```

Handling asynchronous exceptions with Active patterns

```
try
  client.ResourceGroups.Delete(groupName)
with
| :? AzureException as e
  when e.code = "ResourceGroupNotFound" ->
  return failwith "Aggregated Not found exception!"
```



Exception not caught!

Because .Net Async Tasks wrap exceptions in ***System.AggregateException***

Active Pattern matching aggregated exceptions

```
(exn -> 'a option) -> System.Exception -> 'a option
let rec (|IsAggregateOf|_) condition (e: System.Exception) =
  match e with
  | :? System.AggregateException as e ->
    Seq.tryPick (IsAggregateOf condition) e.InnerExceptions
  | e -> condition e
```

Matching condition

```
System.Exception -> string option
let SomeResourceGroupNotFoundException (e: System.Exception) =
  match e with
  | :? AzureException as e
    when e.code = "ResourceGroupNotFound" -> Some e.code
  | _ -> None
```

Exception properly caught!

```
try
  client.ResourceGroups.Delete(groupName)
with
| IsAggregateOf SomeResourceGroupNotFoundException e ->
  return failwith "Aggregated Not found exception!"
```

Port to F# stories

PowerShell scripts

statically checked, strongly typed, **code 2.73 smaller**

Perl scripts

37% smaller, more readable, better logging (.Net), ETW telemetry, testable, ...

C# frontend

Web, Database Abstraction Layer (EF)
40% smaller (thousands LOC)

Testimonials from Springfield dev team

- *“F# equipped me with new tools to see and decompose problems in new ways.”*
- *“F# changed the way I look at solving problems and writing code (...) allows me to start thinking in code“*
- *“Refactoring became an enjoyable process.”*

F# at Microsoft

PRODUCT TEAMS

Bing: ad sales simulation

Xbox Live: Matchmaking

Windows: Compatibility Testing & Tools

Azure Batch

PRODUCT SUPPORTING F#

Visual Studio, Azure Notebooks, Azure Function

RESEARCH

Program verification: termination analysis (SLAM),
memory safety checker (SLayer), verification language
(F*)

Quantum Computing: Liquid tool suite

Bioinformatics: DNA and genetic programming (DSD, GEC,
SPiM)

Distributed Computing/Big Data: Prajna

Free
fsharp.org
stickers!

Appropriate
use of C++



Thanks

- Lambda Days organizers
- Springfield Team at Microsoft
- Phillip Carter (F# team)



Relevant Links

<https://blogs.msdn.microsoft.com/dotnet/2016/12/13/project-springfield-a-cloud-service-built-entirely-in-f/>

<http://fsharpforfunandprofit.com/posts/conciseness-intro/>

<http://blumu.github.io/ResumableMonad/>

<https://azure.microsoft.com/en-us/blog/scaling-up-project-springfield-using-azure/>

<https://github.com/Microsoft/fsharplu/wiki>

Backup slides

Features coming in VS 2017

- Tooling support in VS (Roslyn)
- Feature parity with C#/VB
- F# the best-tooled functional programming language in the market
- .Net Core Linux support

F# tool to reduce beta-redex with traversals

<http://william.famille-blum.org/research/tools.html>

